AWS S3 | Node.js | React

AWS S3 File Upload System

Full-Stack Cloud Application

React Frontend · Node.js Backend · AWS S3 Storage

A production-ready full-stack application that enables seamless file uploads and downloads through AWS S3. Built with a React frontend, Express/Node.js backend, and AWS SDK v3 — featuring CORS support, Multer file handling, environment-based configuration, and a clean REST API.

Layer	Technology	Purpose
Frontend	`React + Vite + Axios`	User Interface & HTTP requests
Backend	`Node.js + Express + Multer`	REST API & file parsing
Cloud Storage	`AWS S3 + SDK v3`	File storage & public URLs
Config	`dotenv (.env)`	Secure credentials management

Project Structure

The project follows a clean separation of concerns with a dedicated backend API server and a React-based frontend. Configuration is managed via environment variables for security.

Directory Layout

```
aws-s3-file-upload-system/
■
■■■ backend/
■ ■■■ server.js # Express server, S3 client, upload route
■ ■■■ package.json # Backend dependencies
■ ■■■ .env.example # Environment variable template
■
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ App.jsx # Main React component
■ ■■■ public/
■ ■■■ package.json # Frontend dependencies
■
■■■ README.md
■■■ .gitignore
■■■ LICENSE (optional)
```

Prerequisites

Requirement	Details
JavaScript / React / Node.js	Basic working knowledge required
AWS Account	With access to create and manage S3 buckets
Node.js installed	v16+ recommended (LTS)
npm / yarn	Package manager for installing dependencies

AWS S3 Bucket Policy

Create an S3 bucket, enable public access, and apply the following bucket policy to allow public read/write operations. Replace **awsmfilebucket** with your actual bucket name.

S3 Bucket Policy (JSON)

```
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "Statement1",
"Effect": "Allow",
"Principal": "*",
"Action": "s3:*",
"Resource": "arn:aws:s3:::awsmfilebucket/*"
}
]
}
```

Backend Setup

Step 1 — Initialize Node.js Project

Terminal

```
mkdir backend && cd backend
npm init -y
npm install express
```

Step 2 — Install Dependencies

Terminal

```
npm install multer @aws-sdk/client-s3 dotenv cors
```

Step 3 — Environment Configuration

Create a **.env** file in the backend directory (copy from **.env.example**) and fill in your AWS credentials:

.env.example

```
# .env.example
PORT=5000
AWS_ACCOUNT_REGION=us-east-1
AWS_ACCOUNT_ACCESS_KEY=your_access_key_here
AWS_ACCOUNT_SECRET_ACCESS_KEY=your_secret_key_here
AWS_BUCKET_NAME=your_bucket_name_here
```

Step 4 — Complete server.js

backend/server.js

```
// server.js
import express from 'express';
import multer from 'multer';
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import dotenv from 'dotenv';
import cors from 'cors';
dotenv.config();

const app = express();
const port = process.env.PORT || 5000;

// Enable CORS
app.use(cors());
```

```
app.use(express.json());

// Create S3 client using env credentials
const s3Client = new S3Client({
region: process.env.AWS_ACCOUNT_REGION,
credentials: {
accessKeyId: process.env.AWS_ACCOUNT_ACCESS_KEY,
secretAccessKey: process.env.AWS_ACCOUNT_SECRET_ACCESS_KEY,
},
});

// Multer: store files in memory, limit 25MB
const storage = multer.memoryStorage();
const upload = multer({
storage,
limits: { fileSize: 25 * 1024 * 1024 },
});

// POST /api/upload — receive file and push to S3
app.post('/api/upload', upload.single('file'), async (req, res) => {
try {
const file = req.file;
const params = {
Bucket: process.env.AWS_BUCKET_NAME,
Key: file.originalname,
Body: file.buffer,
ContentType: file.mimetype,
};
await s3Client.send(new PutObjectCommand(params));
const fileUrl = `https://${process.env.AWS_BUCKET_NAME}` +
`.s3.amazonaws.com/${file.originalname}`;
res.status(200).json({ fileUrl });
} catch (error) {
console.error('Failed to upload file to S3:', error);
res.status(500).json({ error: 'Failed to upload file to S3' });
}
});

app.listen(port, () => {
console.log(`Server started on port ${port}`);
});
```

Frontend Setup

Step 1 — Scaffold React App with Vite

Terminal

```
mkdir frontend && cd frontend
npm create vite@latest ./
# Choose: React > JavaScript
```

Step 2 — Install Axios

Terminal

```
npm install axios
npm run dev # Start development server on http://localhost:5173
```

Step 3 — Complete App.jsx

frontend/src/App.jsx

```
// frontend/src/App.jsx
import { useState } from 'react';
import axios from 'axios';

function App() {
const [selectedFile, setSelectedFile] = useState(null);
const [uploadedFileUrl, setUploadedFileUrl] = useState('');

const handleFileChange = (event) => {
setSelectedFile(event.target.files[0]);
};

const handleUpload = async () => {
try {
const formData = new FormData();
formData.append('file', selectedFile);
const response = await axios.post(
'http://localhost:5000/api/upload',
formData,
{ headers: { 'Content-Type': 'multipart/form-data' } }
);
setUploadedFileUrl(response.data.fileUrl);
```

```
} catch (error) {
console.error('Failed to upload file:', error);
}
};
const handleDownload = () => {
if (uploadedFileUrl) window.open(uploadedFileUrl, '_blank');
};
return (
<div className="container">
&lt;h1&gt;File Upload and Download&lt;/h1&gt;
<p>Select a file:</p>
<input type="file" onChange={handleFileChange} />
<button
disabled={!selectedFile}
onClick={handleUpload}
>
Upload
</button>
{uploadedFileUrl && (
<div>
<p>File uploaded!</p>
<button onClick={handleDownload}>Download</button>
</div>
)}
</div>
);
}
export default App;
```

Package Configuration

backend/package.json

backend/package.json

```
{
"name": "backend",
"version": "1.0.0",
"type": "module",
"scripts": {
"start": "node server.js",
"dev": "nodemon server.js"
},
"dependencies": {
"@aws-sdk/client-s3": "^3.x.x",
"cors": "^2.x.x",
"dotenv": "^16.x.x",
"express": "^4.x.x",
"multer": "^1.x.x"
}
}
```

frontend/package.json

frontend/package.json

```
{
"name": "frontend",
"version": "0.0.0",
"type": "module",
"scripts": {
"dev": "vite",
"build": "vite build",
"preview": "vite preview"
},
"dependencies": {
"axios": "^1.x.x",
"react": "^18.x.x",
```

```
"react-dom": "^18.x.x"
},
"devDependencies": {
"@vitejs/plugin-react": "^4.x.x",
"vite": "^5.x.x"
}
}
```

Application Data Flow

Step	Actor	Action	Result
1	User	Selects a file via	React state updated with File object
2	React	Calls handleUpload(), builds FormData	Axios POST to /api/upload
3	Express	Multer parses multipart request	File stored in memory buffer
4	Express	Calls s3Client.send(PutObjectCommand)	File streamed to AWS S3
5	AWS S3	Stores file in configured bucket	Returns success acknowledgement
6	Express	Constructs public S3 URL	Returns { fileUrl } as JSON
7	React	Stores fileUrl in state	Shows Download button to user
8	User	Clicks Download	Opens S3 URL in new tab

Project in Action

The screenshot below shows the working application: AWS S3 bucket with uploaded files (top), the Node.js backend server running in terminal (middle), and the React frontend UI with the File Upload & Download interface (bottom left).

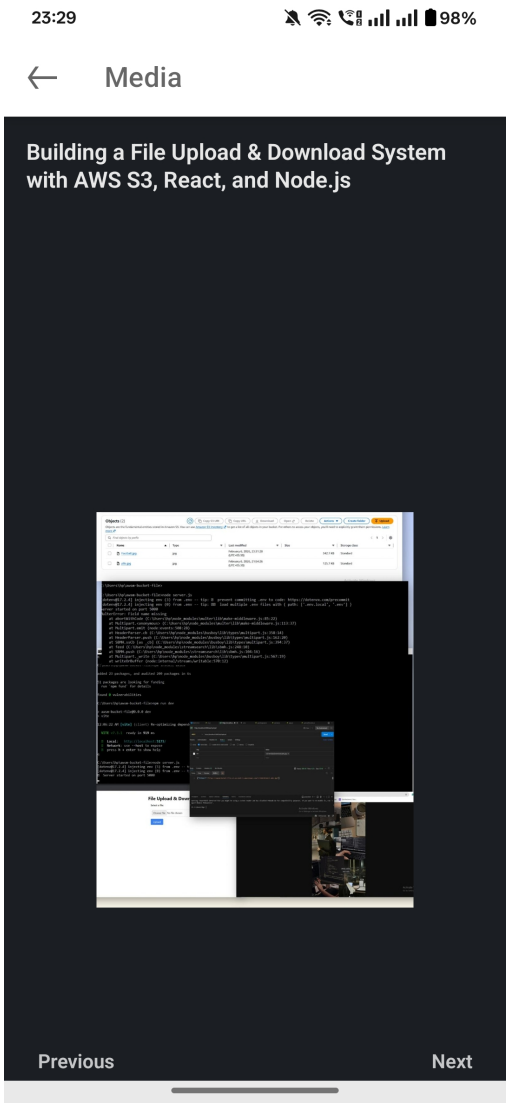

Figure 1: AWS S3 bucket objects view, Node.js server terminal output, and React frontend UI

Security Considerations

■ **Never commit .env files**	Add .env to .gitignore. Use .env.example as a template for other developers.
■ **Restrict Bucket Policy**	The sample policy uses s3:* for all principals. In production, scope permissions tightly — e.g., separate IAM roles for read vs. write.
■ **File Type Validation**	Add Multer fileFilter to allow only specific MIME types (e.g., image/*, application/pdf) and reject dangerous file types.
■ **File Size Limits**	The backend enforces a 25MB limit via Multer. Adjust based on your use case and S3 pricing.
■ **Unique File Keys**	Using file.originalname as the S3 key can cause overwrites. Use uuid or timestamps to generate unique keys.
■ **CORS Configuration**	Restrict CORS origin in production to your frontend domain instead of the wildcard default.

Suggested Enhancements

Enhancement	Description
File Type & Size Validation	Use Multer's fileFilter to validate MIME types and reject uploads server-side.
Unique S3 Keys with UUID	npm install uuid, then set Key: `${uuidv4()}-${file.originalname}` to prevent overwrites.
Upload Progress Bar	Use Axios onUploadProgress callback to show a real-time progress bar in the UI.
List & Delete Files	Add GET /api/files (ListObjectsV2Command) and DELETE /api/files/:key (DeleteObjectCommand) routes.

Presigned URLs	Use GetObjectCommand with getSignedUrl() for time-limited, secure download links instead of public bucket access.
Authentication	Add JWT-based auth or AWS Cognito to restrict uploads to authenticated users only.
Error Feedback in UI	Display user-friendly error messages in React when upload fails, rather than logging to console.
CI/CD Deployment	Containerize with Docker and deploy backend to ECS/Fargate; deploy frontend to S3 + CloudFront.